When Generated Answers Become Evidence: Error Propagation in Human-Gated Self-Updating Retrieval-Augmented Generation

Pavan Kumar Guthikonda *pavansky143@gmail.com*
Independent researcher

Abstract

Retrieval-augmented support systems store the answers their users accept, so that later questions can be answered from them. A wrongly accepted answer then becomes evidence, and answers generated from it can be accepted and stored in turn. We measure this human-gated loop on real repeat questions from four CQADupStack forums, with community duplicate links as ground truth, three retrievers, 48,026 generated answers each labelled by an independent judge, and a simulated user who accepts wrong answers at a swept rate. Write-back can fix only 3% to 20% of questions, and it fixes a few percent of those while breaking a similar share of correct answers when users accept most wrong answers. With content held fixed, stored generated answers displace the correct thread more often than the human answers they paraphrase, but storing the question with the answer removes most of this source bias and the outcome barely changes. Along chains of answers generated from answers, the share that still states the original solution falls from 76% to 58%, yet deep entries add almost no net harm within the stream. That containment depends on the generator being told to stay within its evidence: with an open prompt the share falls from 83% to 47% and on average entries of every depth cost correct answers. Neither the first-order crossover estimate of earlier work nor a depth-aware one predicts the crossover out of sample. Reopening wrong answers, even one round late or with false reopens, removes much of the loss; a depth cap and regeneration from the source thread do not improve on naive write-back, and down-weighting stored entries, uniformly or by provenance depth, removes the benefit instead.

1 Introduction

Support systems built on retrieval-augmented generation (RAG) (Lewis et al., 2020) increasingly learn from their own use. When a user accepts an answer, the question and the generated answer are written back into the knowledge base, and the next similar question can retrieve that entry and generate from it (Liu et al., 2026; Wang et al., 2026a; Consortium for Service Innovation, 2016). The loop is attractive because repeat questions are common, and it is risky because users accept wrong answers. A wrongly accepted answer becomes evidence, and an answer generated from it can be accepted and stored in turn, so an error can travel along a chain of answers that were each generated from the previous one.

Two strands of work suggest the risk is real. Neural retrievers prefer model-written text (Dai et al., 2024b), and generated text written back into a corpus displaces human documents over repeated rounds (Chen et al., 2024a; Yu et al., 2026; Druck & Smith, 2026). Agents that store their own outputs propagate their errors (Xiong et al., 2026; Wang et al., 2026b; Chen et al., 2026). But these studies either add generated text without any gate, or gate it with an oracle that admits only correct outputs, and none has ground truth for whether a later question was answered correctly. A deployed support system sits in between: a human gate that lets wrong answers through at some rate, and real repeat questions whose correct answers are known.

An earlier study measured this human-gated loop when the stored entry is only a pointer to an existing thread, and found that write-back helps while users reject most wrong answers and hurts beyond a crossover

acceptance rate (Guthikonda, 2026). With pointer entries no text is generated, so errors cannot change an answer's content or travel through chains. Here we store what deployed systems store, the generated answer, and ask four questions. Do retrievers prefer the stored model-written answers over human text with the same content (H1)? Do errors propagate along chains of answers generated from answers (H2)? Can the acceptance rate at which write-back turns harmful be predicted when entries have depth (H3)? And which countermeasures remove the harm without removing the benefit (H4)?

We answer them on four CQADupStack forums with three retrievers, ten random orders and real repeat questions, generating 48,026 answers and labelling every one with an independent judge. Four design choices make the measurement hard to explain away. A same-content control replaces each generated answer by the human answer it paraphrases, cut to the same length, so source bias is tested with content held fixed. A content-aware user accepts according to what the answer says, not only where it came from, so text that loses its solution counts as wrong. Effects are measured against the questions write-back could possibly fix, with uncertainty from a bootstrap over both orders and duplicate families. And countermeasures are tested in realistic form, with delayed and mistaken reopening and rules that act on provenance depth.

Our findings are:

- **The stakes are small but two-sided.** Only 3% to 20% of questions can be fixed by write-back, and it fixes a few percent of them while breaking a similar share of correct answers when users accept most wrong answers (Section 4.1).
- **Source bias exists inside the loop but changes little.** Stored generated answers displace the correct thread more than length-matched human answers when the answer is stored alone, but storing the question with it removes most of the difference, and outcomes barely move (Section 4.2).
- **Text degrades along chains; harm does not cascade, as long as the generator stays close to its evidence.** Answers generated from generated answers state the original solution markedly less often, but deep entries add almost no net harm within the stream. Without the evidence-only instruction the text degrades about twice as steeply and entries start to cost correct answers on average (Sections 4.3 and 4.6).
- **The crossover is not predictable out of sample for generated entries**, by the earlier first-order estimate or by a depth-aware one (Section 4.4).
- **Reopening wrong answers works even when it is late or noisy; rules that act on provenance depth do not beat it** (Section 4.5).

All code, every generated answer and every judge label are released.

2 Related work

Generated text in retrieval corpora. Neural retrievers rank text written by language models above human text with the same content (Dai et al., 2024b), and paired benchmarks of human passages and model rewrites now measure this source bias (Dai et al., 2024a). Later work traces the bias to low perplexity (Wang et al., 2025), to relevance supervision (Xion & Nejdl, 2026; Huang et al., 2026) and to term statistics in lexical rankers (Huang et al., 2025), and readers show a matching preference for generated contexts (Tan et al., 2024). Once generated text accumulates it displaces human evidence. Chen et al. (2024a) index the answers of a RAG system back into an open-domain corpus for ten rounds and find human passages pushed out of the top ranks, Yu et al. (2026) find that retrieval exposure turns synthetic while answer accuracy looks stable, and Druck & Smith (2026) show that a model retrieving references it wrote itself collapses in most simulations because it over-cites its own text, a bias that survives quality controls but that their observational design cannot separate from content. Write-back of generated answers has also been proposed as an improvement behind automatic validation (Chinthala, 2025) or a reference-based quality score in community QA (Chen et al., 2025). Our loop differs in that generated text enters only after a simulated user accepts it at a false-acceptance rate we sweep, every stored entry records its provenance depth and, when

retrieved, is the only evidence for the next answer, and the human-answer control with the same question, root thread, length and position is applied inside the loop rather than to a static corpus.

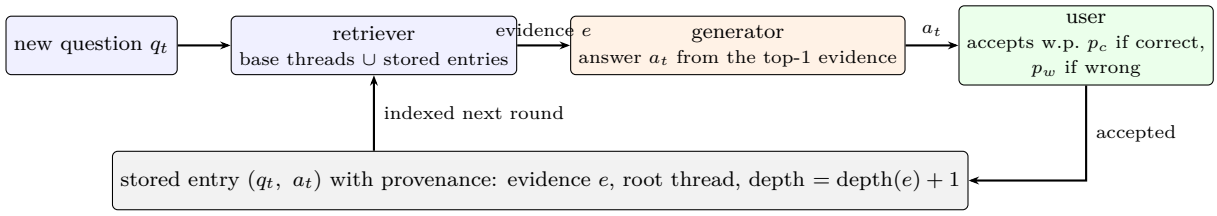
Feedback loops and model collapse. Training generative models on their own outputs erodes the tails of the distribution (Shumailov et al., 2024) and degrades quality or diversity unless fresh real data enters each generation (Alemohammad et al., 2024), whereas accumulating synthetic data beside real data rather than replacing it avoids collapse (Gerstgrasser et al., 2024). When people curate the synthetic samples before retraining, the loop provably optimizes their preference (Ferbach et al., 2024), so a human filter selects for what is accepted rather than for what is correct. Model outputs that re-enter training data amplify dataset biases (Taori & Hashimoto, 2023), predictions that shift the distribution they predict are performative (Perdomo et al., 2020), and recommender loops homogenize behaviour (Chaney et al., 2018), degenerate user interests (Jiang et al., 2019) and escalate a preference for generated content through user clicks (Zhou et al., 2025). A stochastic model of how generated answers are produced and indexed warns that adoption can outpace human verification (Garetto et al., 2025), and activity on Stack Overflow has fallen since ChatGPT was released (del Rio-Chanona et al., 2024), so forum knowledge bases may increasingly hold generated answers. These loops act through retraining, preferences or an open pool of documents. In ours the weights stay fixed, accepted entries accumulate beside the original threads, and acceptance is the only filter, so community duplicate labels let us measure how a wrongly accepted answer is reused as evidence and how far its descendants reach along a provenance chain.

Agent memory and error propagation. Within a single response an early mistake leads a model into further mistakes (Zhang et al., 2024), and agents that store their own outputs carry such errors across tasks. Xiong et al. (2026) show that agents copy the records they retrieve, that stored errors propagate, and that selective addition through a simulated human oracle, which admits only correct executions, improves performance. Errors in stored rules and summaries reproduce themselves through retrieval (Wang et al., 2026b), write-back lets one source fill several retrieval slots through records that point to their parents (Li et al., 2026), and an agent writing to an append-only store either cleans its errors or is captured by them, with majority admission arming a false majority and provenance marks helping only when they track truth (Chen et al., 2026). Proposed remedies include admission by self-consistency (Zhang & Li, 2026), screening and provenance ranking, which Karunanidhi (2026) find of limited use, and dependency-aware cleanup (Choudhary, 2026), while attacks show that injected records steer later outputs (Zou et al., 2025; Chen et al., 2024b; Dong et al., 2025). These studies use agent benchmarks or adversaries, and their gates are either oracles or automatic. We study a support knowledge base whose benign but imperfect user lets wrong answers through at a controlled rate, record the depth of every entry, and compare countermeasures that act on depth, namely a depth cap, regeneration from the root thread and depth-scaled down-weighting, with reopening and uniform down-weighting.

The pointer-only loop. Guthikonda (2026) measured the same human-gated loop on all twelve CQADupStack forums with entries that point to an existing thread and generate no text. Write-back helped while users rejected most wrong answers and hurt beyond a crossover acceptance rate, the harm came from question-to-question similarity, and a first-order estimate from per-entry effects predicted the crossover on held-out orders. We keep that protocol, ground truth and estimate, so the two studies are directly comparable, and change the one thing deployed systems do differently: the stored entry is a generated answer that becomes the evidence for the next one.

3 Setup

Data and stream. We follow the protocol of Guthikonda (2026) so that every number is comparable with the pointer-only loop studied there. We use four CQADupStack forums (Hoogeveen et al., 2015) in their BEIR form (Thakur et al., 2021): android, unix, webmasters and mathematica. The knowledge base is the forum's set of threads. The incoming questions are the held-out questions whose community duplicate links point into the knowledge base, between 1,358 and 1,696 per forum, and questions that share a duplicate target form a family. Families are what makes write-back useful: a later question can only be helped by an entry written for an earlier member of its own family. Between 53% and 73% of questions belong to a family



An entry generated from a thread has depth 1. An entry generated from another entry has depth 2 or more, and its evidence chain ends at a root thread. Root correctness asks whether the root thread is a community-marked duplicate of q_t; content correctness additionally asks an independent judge whether a_t states the root thread's solution.

Figure 1: The loop we simulate. Only the user's accept decision is simulated; retrieval, generation and judging are real.

with more than one question, and the largest family holds up to 15% of a forum's questions, which is why we also report results without it. Each base thread is completed with its accepted answer, or failing that its top-voted answer, fetched by post id from the public Stack Exchange API. Questions arrive in ten random orders. Each order is split into ten rounds, and an entry written in one round becomes retrievable in the next, as in a knowledge base that is re-indexed in batches.

Retrieval and generation. Three retrievers score base threads and stored entries together: bge-small-en-v1.5 (Xiao et al., 2024), all-MiniLM-L6-v2 (Wang et al., 2020; Reimers & Gurevych, 2019) and BM25 (Robertson & Zaragoza, 2009) with the base corpus statistics. The top-1 item is the evidence. If it is a thread, the generator sees the thread and its accepted answer. If it is a stored entry, the generator sees the stored question and the stored answer and never the root thread, as in a deployed system that retrieves its own past answers. The generator is `gpt-5.4-mini` with minimal reasoning and a fixed seed, instructed to answer from the evidence only in at most 120 words. Prompts rejected by the provider's content filter, 1 in total, were answered by an open-weight model and are marked in the released cache. Every answer is generated once per (question, evidence) pair and cached, so every condition that reaches the same pair sees the same text. The loop of generate, judge, index and simulate was iterated until no served pair was missing, which took 48,026 generated answers.

What is stored. An accepted answer is written back as an entry that carries its provenance: the evidence it was generated from, the root thread at the end of that evidence chain, and its depth (Figure 1). We compare three entry designs. The *generated answer* alone; the *question with the generated answer*, the realistic self-written question and answer pair and our main design; and the *generated answer with the root thread's text*, which stores the answer together with its cited source. To test source bias with content held fixed, we add two controls built from the same keys. For every entry, the control replaces the generated answer by the root thread's human answer, cut to the same number of words, with or without the question. Each generated design and its control therefore share the question, the root, the length and the position in the stream, and differ only in who wrote the answer. As further reference points we show the two pointer designs of Guthikonda (2026), which store the question alone or the question with the thread text and generate nothing.

Correctness and the user. An answer is *root-correct* when the root thread of its evidence chain is a community-marked duplicate of the question. This is the label of Guthikonda (2026). It ignores the text of the answer, so on its own it cannot show an error that the generated text introduces. We therefore also label every generated answer with an independent judge, `DeepSeek-V4.1-Flash`, which reads the question, the root thread's human answer and the generated answer and replies YES, PARTLY, NO or ABSTAIN. An answer is *content-correct* when it is root-correct and the judge says YES. When the root thread has no human answer on record, which is the case for 12% of served answers, content-correctness falls back to root-correctness. The judge labelled every generated answer whose root thread has a human answer, 42,497 in all. On a separate audit of 900 answers it agrees with the two judges described in Appendix C on 81%

and 75% of answers, which is higher than their 78% agreement with each other. The simulated user accepts a correct answer with probability $p_c = 0.9$ and a wrong one with probability p_w, the swept parameter. The *content-aware* user, our main user model, applies this to content-correctness, so an answer from the right thread that lost its solution is treated as a wrong answer. The *root-label* user applies it to root-correctness, as in the earlier paper.

Countermeasures. We test rules a deployed system could apply. *Reopen* removes a wrongly accepted answer's entry with probability 0.5. In the oracle form of the earlier paper the entry is removed before it is ever retrievable, which is equivalent to filtering half of the wrong entries at write time. We therefore add realistic forms in which the entry is retrievable for one or two rounds before it is reopened, and forms in which 10% or 20% of correct entries are reopened by mistake. *Depth cap* stores only answers generated from a base thread, so no entry of depth 2 or more exists. *Regenerate from root* answers from the root thread of a retrieved entry rather than from the entry's text, so the text is always one step from a human answer. *Down-weighting* subtracts λ times the interquartile range of static top-1 scores from every entry's score, either uniformly or multiplied by the entry's depth, with $\lambda \in \{0.5, 1\}$.

Measures and statistics. Δ is the change in the share of questions answered correctly over the stream against static retrieval, in percentage points, using content-correctness for the content-aware user and root-correctness for the root-label user. The crossover p^* is the smallest p_w at which the mean Δ turns negative, by linear interpolation on the grid $p_w \in \{0.1, 0.2, \ldots, 0.9, 0.95\}$. A configuration is *safe* when Δ stays non-negative up to $p_w = 0.95$ and *robustly safe* when the lower end of the crossover's bootstrap interval is also above 0.95. Because Δ averages over all questions, we also measure it against what write-back could achieve. A question is *fixable* when static retrieval answers it wrongly and an earlier member of its family was answered correctly, so that a correct entry for it could exist. We report the share of fixable questions that write-back fixes, the share of statically correct questions it breaks, and the net change in correct answers per 1,000 questions. Intervals for Δ come from a joint bootstrap over orders and families, which accounts for the dependence between questions of the same family. Comparisons between conditions are paired by order, use two-sided paired t-tests, and are corrected for the twelve forum–retriever configurations with the Holm procedure.

Hypotheses. We organised the study around four hypotheses written down before the first simulation. Appendix A lists every way in which the analysis departs from that plan, including changes made after an internal code review. (H1) Entries written by the model displace the correct human thread more often than entries with the same content written by people. (H2) Errors propagate through provenance chains: entries generated from entries are served increasingly often, carry less of the original solution, and cause harm beyond that of first-level entries. (H3) A crossover estimate that accounts for depth predicts p^* on held-out orders better than the first-order estimate of Guthikonda (2026). (H4) Countermeasures differ in how much loss they remove at high p_w per unit of gain they give up at low p_w, and rules that act on provenance do better than rules that do not.

4 Results

4.1 How much is at stake

Write-back can only help a question whose family has already been answered correctly, so the room for improvement is small and differs widely between configurations: between 3% and 20% of questions are fixable (Table 1, Figure 2). With the realistic entry design and the content-aware user, write-back fixes on average 3.7% of the fixable questions when users accept 10% of wrong answers and 3.5% when they accept 95%. It breaks 0.5% and 2.3% of the questions static retrieval answered correctly. The net change ranges from +0.0 to +5.7 correct answers per 1,000 questions at $p_w = 0.1$ and from -1.9 to +2.1 at $p_w = 0.95$. The gain is significantly positive in 6 of 12 configurations at $p_w = 0.1$, and at $p_w = 0.95$ the interval includes zero in 12 of 12. The changes are small because most questions are not fixable and because write-back recovers only a few percent of those that are, not because the loop is inert: in every configuration it both fixes and

Table 1: How much is at stake, content-aware user, question and generated answer entries. Fixable: share of questions that static retrieval answers wrongly and that have an earlier, correctly answered member of their family. Fixed: share of fixable questions answered correctly with write-back. Broken: share of statically correct questions answered wrongly. Net: change in correct answers per 1,000 questions. Δ: points, 95% interval from a joint bootstrap over orders and families; the last column drops the largest family.

forum	retriever	static	fixable	fixed 0.1 / 0.95	broken 0.95	net 0.95	Δ at 0.95	without largest
android	bge	15.6%	16.4%	4.2 / 3.9%	3.4%	+1.5	+0.15 [-0.23, +0.57]	+0.17 [-0.30, +0.64]
android	MiniLM	20.3%	19.6%	3.5 / 3.0%	2.2%	+1.4	+0.14 [-0.32, +0.49]	+0.06 [-0.40, +0.48]
android	BM25	12.6%	13.6%	1.2 / 1.4%	1.1%	+1.8	+0.18 [-0.09, +0.48]	+0.21 [-0.08, +0.52]
unix	bge	19.8%	7.5%	6.8 / 6.9%	1.7%	+2.1	+0.21 [-0.12, +0.54]	+0.23 [-0.10, +0.57]
unix	MiniLM	22.3%	8.3%	5.2 / 4.9%	2.0%	+0.9	+0.09 [-0.33, +0.50]	+0.16 [-0.21, +0.51]
unix	BM25	13.4%	5.0%	2.9 / 2.7%	1.7%	-0.1	-0.01 [-0.27, +0.25]	-0.01 [-0.26, +0.26]
webmasters	bge	10.5%	9.9%	2.6 / 2.1%	3.7%	-1.9	-0.19 [-0.56, +0.12]	-0.23 [-0.67, +0.10]
webmasters	MiniLM	11.3%	17.1%	2.2 / 2.1%	1.9%	+1.9	+0.19 [-0.15, +0.61]	+0.22 [-0.19, +0.68]
webmasters	BM25	8.7%	17.8%	0.8 / 0.9%	0.6%	+1.6	+0.16 [-0.02, +0.41]	+0.19 [-0.03, +0.47]
mathematica	bge	11.6%	6.8%	1.1 / 0.8%	2.5%	-1.7	-0.17 [-0.42, +0.06]	-0.18 [-0.46, +0.05]
mathematica	MiniLM	12.2%	4.2%	9.1 / 7.6%	4.3%	-1.5	-0.15 [-0.53, +0.19]	-0.16 [-0.53, +0.21]
mathematica	BM25	9.0%	2.9%	5.2 / 5.3%	3.2%	-0.7	-0.07 [-0.39, +0.23]	-0.08 [-0.37, +0.24]

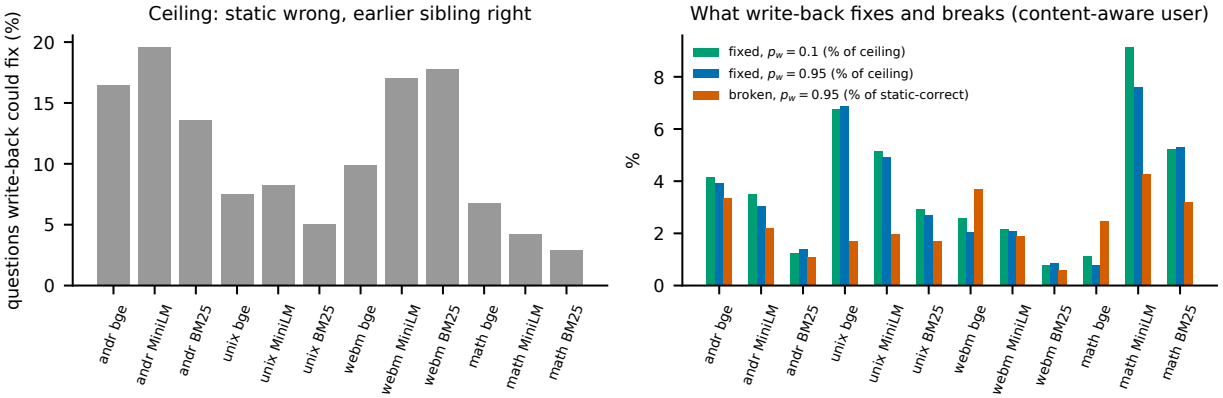


Figure 2: Left: share of questions that write-back could fix. Right: share of those it fixes at low and high acceptance of wrong answers, and share of statically correct questions it breaks at high acceptance. Content-aware user, question and generated answer entries.

breaks answers, and the two nearly cancel at high p_w. Dropping the largest family, which holds up to 15% of a forum's questions, does not change any of these conclusions (last column of Table 1).

4.2 H1: do retrievers prefer stored model-written answers?

Figure 3 shows Δ against p_w for every configuration. Entries that store the question with the generated answer are safe up to $p_w = 0.95$ in 7 of 12 configurations for the content-aware user, robustly so in 4; the question-only pointer entries of the earlier paper are safe in 3 and the question-with-thread pointer entries in 4. Steal rates follow the same order: the median share of statically correct answers displaced by an entry at $p_w = 0.95$ is 3.1% for question and generated answer, 3.2% for question and thread, and 12.7% for the question alone.

These comparisons differ in content as well as authorship. The control holds content fixed (Table 2, Figure 4). When the entry is the answer alone, generated answers displace the correct thread more often than the human answers they paraphrase: the steal rate is higher in 10 of 12 configurations, significantly so in 5 after Holm correction and significantly lower in none, by +1.5 points for bge, +1.2 for MiniLM and +1.0 for BM25. This is the source bias reported for static corpora (Dai et al., 2024b), now measured inside the loop, and it appears for BM25 as well, which is consistent with the term-statistics account of Huang et al. (2025).

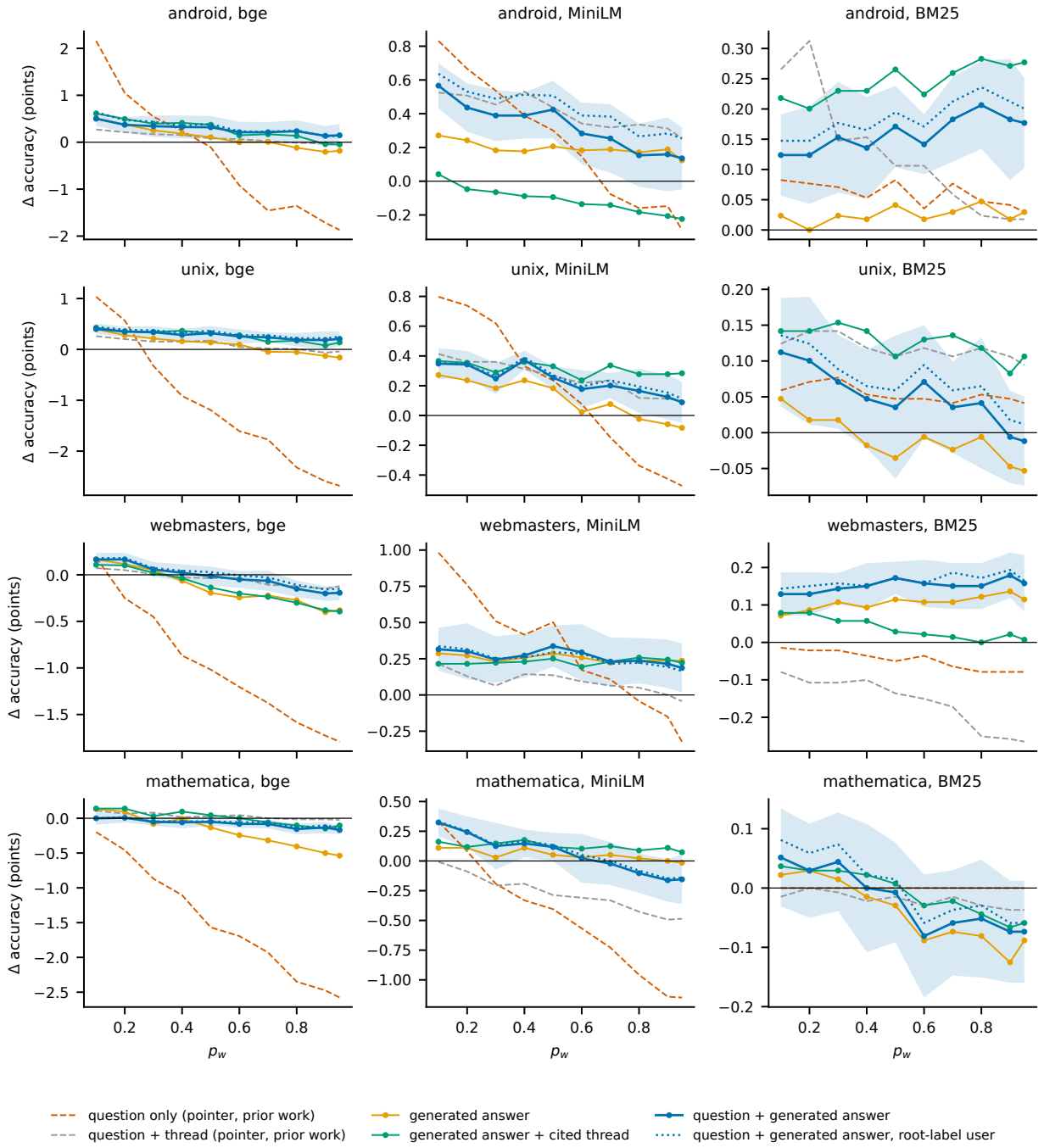


Figure 3: Δ against the acceptance rate of wrong answers, by forum and retriever. Solid: generated-answer entry designs, content-aware user, with a 95% band for question and generated answer. Dotted: the same design with the root-label user. Dashed: pointer designs of the earlier paper.

When the question is stored with the answer, the difference shrinks to -0.7, +0.7 and +1.3 points and is significant in 2 configurations in one direction and 1 in the other. In neither design does authorship change the outcome much: the paired difference in Δ at $p_w = 0.95$ is below a tenth of a point on average. H1 is therefore supported for answer-only entries in its direction but not in its consequence, and storing the question with the answer removes most of it.

Table 2: Source-bias control, root-label user, $p_w = 0.95$. Each generated design is compared with the same entries carrying the root thread's human answer cut to the same length. Steal: correct static top-1 displaced by an entry; capture: an entry outscores the correct thread; both in %. Differences are generated minus human, averaged over the four forums; in brackets, configurations where the paired difference over orders is significant after Holm correction, positive / negative / total.

design pair	retriever	steal (gen.)	steal (human)	Δsteal	Δcapture	Δ (points)	sig. Δsteal
question + answer	bge	3.9	4.6	-0.7	-1.1	+0.08	0/1/4
question + answer	MiniLM	3.8	3.1	+0.7	+0.0	+0.04	0/0/4
question + answer	BM25	2.7	1.4	+1.3	+2.9	-0.01	2/0/4
answer only	bge	8.6	7.1	+1.5	+3.0	+0.03	1/0/4
answer only	MiniLM	3.0	1.8	+1.2	+3.1	+0.09	2/0/4
answer only	BM25	2.4	1.3	+1.0	+4.4	+0.07	2/0/4

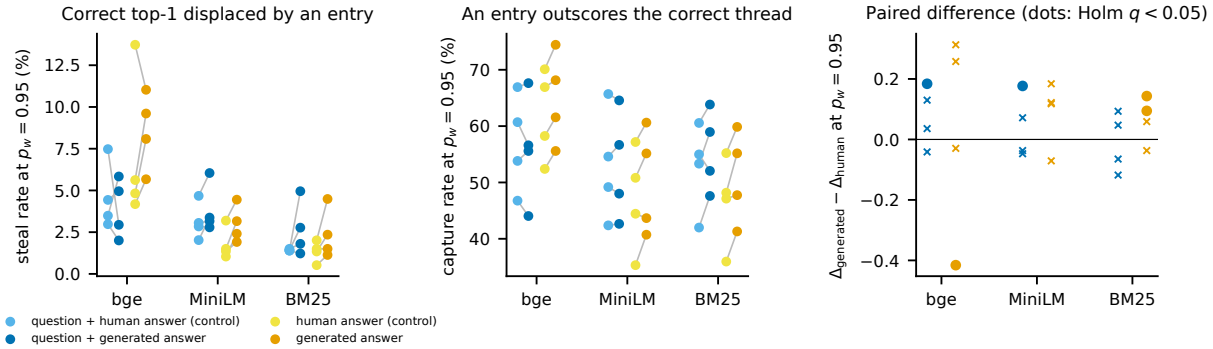


Figure 4: Source-bias control at $p_w = 0.95$. Each line joins a generated design (right point) and its length-matched human control (left point) in one forum. Right panel: paired difference in Δ; filled dots are significant after Holm correction.

4.3 H2: do errors propagate along provenance chains?

Chains form. At $p_w = 0.95$, 5% to 13% of entries were generated from another entry, and the share of answers served from such entries is still rising at the end of the stream, reaching 4.9% in the last round (Figure 5, left). The text degrades along the chain. Across all 1,616 answers served from an entry of depth 2 or more, the judge finds the root thread's solution in 58%, against 62% for depth 1 and 76% for answers generated directly from a thread ($p < 0.001$; Figure 5, middle). The three-judge audit agrees in direction (Appendix C), and similarity to the human answer falls from 0.85 to 0.82 and 0.80 by depth, where an unrelated human answer scores about 0.60.

What does not follow is a cascade of harm. Per 1,000 questions, answers served from depth-1 entries change the number of correct answers by +0.37 on average relative to static retrieval, depth-2 entries by +0.06 and deeper entries by -0.01 (Figure 5, right). Wrong answers served from depth-2 entries are 0.3% to 2.5% of all wrong answers, against 4.6% to 11.6% for depth 1. Two filters explain the gap. A deep entry exists only if a user accepted an answer that was itself served from an entry, and it is served only when it outranks both the base threads and the shallower entries it came from. H2 is supported for the text and for the growth of chains, and not supported for harm within the horizon we can observe.

4.4 H3: can the crossover be predicted?

The first-order estimate of Guthikonda (2026) predicts p^* from the static accuracy and the average effects of a correct and a wrong entry measured at one reference p_w. Our depth-aware estimate keeps the per-entry effects separate for depth-1 and deeper wrong entries and lets the share of deep wrong entries grow

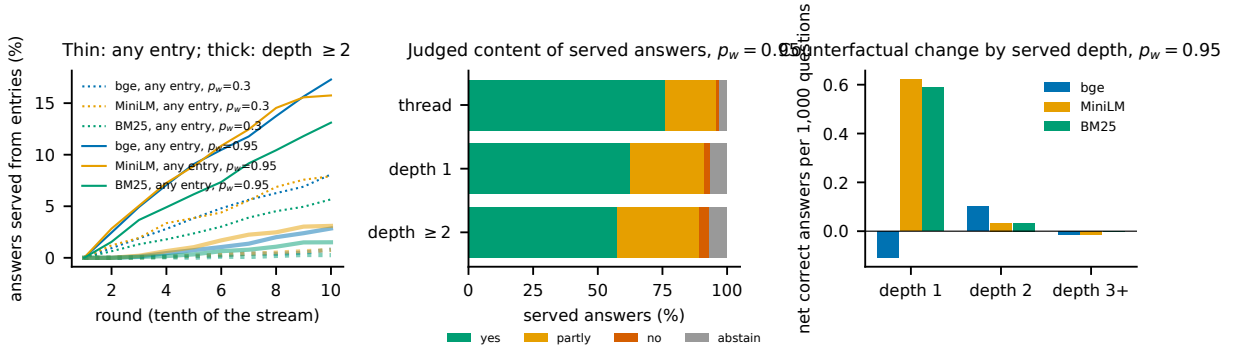


Figure 5: Depth, content-aware user, question and generated answer entries. Left: share of answers served from any entry (thin) and from an entry of depth ≥ 2 (thick), mean over forums. Middle: judged content of served answers by the depth of their evidence, pooled. Right: counterfactual change in correct answers per 1,000 questions attributable to answers served at each depth.

Table 3: Predicting the crossover on held-out orders (calibrate on five orders, evaluate on the other five, both directions; root-label user). MAE over configuration–split pairs where both the observed and the predicted crossover lie inside the grid, with a bootstrap 95% interval. Mean baseline: predicting every observed crossover by their mean. Paired: the two estimates on the pairs where both are defined, with a Wilcoxon signed-rank test.

design	p_{ref}	n	first-order MAE	depth-aware MAE	mean baseline	Spearman (first-order)	Wilcoxon p
question + answer	0.3	6	0.249 [0.157, 0.334]	0.207 [0.114, 0.306]	0.097	-0.43	0.12
question + answer	0.5	6	0.152 [0.082, 0.237]	0.137 [0.059, 0.229]	0.097	0.03	0.09
answer only	0.3	14	0.203 [0.129, 0.290]	0.198 [0.117, 0.288]	0.213	0.33	0.86
answer only	0.5	13	0.165 [0.100, 0.248]	0.157 [0.100, 0.234]	0.221	0.64	0.76
answer + thread	0.3	7	0.217 [0.164, 0.266]	0.223 [0.135, 0.311]	0.155	-0.14	0.08
answer + thread	0.5	6	0.133 [0.035, 0.238]	0.139 [0.045, 0.234]	0.151	0.43	0.16

in proportion to p_w, since each further level needs one more wrong acceptance. At the reference point both estimates coincide; they differ only in how they extrapolate. Table 3 and Figure 6 evaluate both on held-out orders. Few configurations have a crossover inside the grid in both halves of the orders (6 configuration–split pairs for question and generated answer at $p_{\text{ref}} = 0.3$), and on those the first-order estimate has a mean absolute error of 0.249 and the depth-aware estimate 0.207 (Wilcoxon $p = 0.12$), while predicting every crossover by the mean of the observed ones gives 0.097. With generated entries, neither estimate is useful out of sample. The per-entry effects are noisier than with pointer entries, because whether an entry helps now depends on its text as well as its root, and crossovers are rarer. H3 is not supported, and we do not recommend either estimate for generated entries.

4.5 H4: which countermeasures work?

Table 4 and Figure 7 compare the rules with the content-aware user. The oracle reopen of the earlier paper is, as noted in Section 3, naive write-back at half the acceptance rate: its Δ curve matches naive write-back at $p_w/2$ within 0.03 points on average. The realistic forms keep most of its benefit. Reopening after one round raises the mean Δ at $p_w = 0.95$ from +0.04 to +0.14 points, is significantly better than naive write-back in 3 configurations and worse in 0, and costs nothing at $p_w = 0.1$ (+0.26 against +0.25). Waiting two rounds gives +0.10, and reopening 10% or 20% of correct entries by mistake gives +0.11 and +0.09. The rules that act on provenance do not do better. Neither the depth cap (+0.04) nor regenerating from the root thread (+0.05) improves on naive write-back, which is what H2 predicts when deep entries carry little harm: removing or repairing them has little to remove. Down-weighting, uniform or by depth, makes write-back

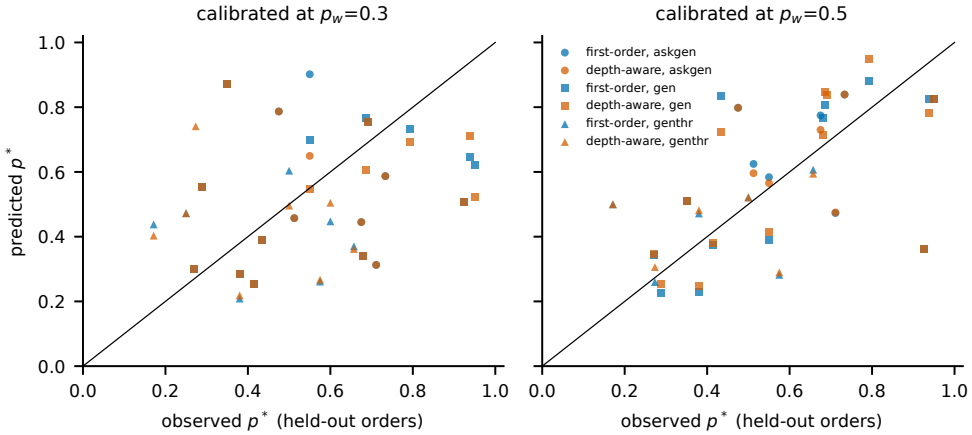


Figure 6: Predicted against observed crossover on held-out orders, for the three generated designs.

Table 4: Countermeasures, content-aware user, question and generated answer entries, mean over the twelve configurations. AUC: mean Δ over the p_w grid. Safe / robust: configurations with no crossover below 0.95, and with the lower bootstrap bound also above 0.95. Better / worse: configurations where the rule is significantly better or worse than naive write-back at $p_w = 0.95$, and significantly worse at $p_w = 0.1$ (paired over orders, Holm-corrected).

rule	Δ at 0.1	Δ at 0.95	AUC	safe / robust	better / worse at 0.95	worse at 0.1
naive write-back	+0.25	+0.04	+0.14	7 / 4	–	–
reopen, oracle (prior work)	+0.26	+0.15	+0.20	9 / 7	5 / 0	0
reopen after 1 round	+0.26	+0.14	+0.19	9 / 7	3 / 0	0
reopen after 2 rounds	+0.26	+0.10	+0.18	8 / 7	3 / 0	0
reopen after 1 round, 10% false	+0.24	+0.11	+0.16	8 / 6	2 / 0	0
reopen after 1 round, 20% false	+0.22	+0.09	+0.14	8 / 5	2 / 0	0
depth cap (depth 1 only)	+0.24	+0.04	+0.13	7 / 5	0 / 0	1
regenerate from root	+0.25	+0.05	+0.14	7 / 6	0 / 1	0
uniform penalty, $\lambda = 0.5$	+0.06	+0.02	+0.04	8 / 5	2 / 2	8
uniform penalty, $\lambda = 1$	+0.01	+0.01	+0.01	10 / 9	2 / 2	9
depth penalty, $\lambda = 0.5$	+0.06	+0.03	+0.04	8 / 5	3 / 2	8
depth penalty, $\lambda = 1$	+0.01	+0.01	+0.01	10 / 9	2 / 2	9

nearly inert: it removes the gain at $p_w = 0.1$ in most configurations (significantly worse than naive in 8 of 12 for the depth penalty with $\lambda = 0.5$) and leaves little to protect at $p_w = 0.95$. H4 is supported in its first part, since the rules differ sharply in what they give up for what they protect, and rejected in its second: no rule that acts on provenance beats delayed, noisy reopen.

4.6 Robustness

User model. Judging what the answer says rather than where it came from lowers the benefit slightly, from +0.29 to +0.25 points at $p_w = 0.1$ and from +0.07 to +0.04 at $p_w = 0.95$, and moves the crossover earlier in 4 configurations (Figure 3, dotted against solid). The conclusions above hold for both users; Appendix D gives every number for both.

Generator and prompt. Figure 8 repeats the main design with an open-weight generator (android) and with a prompt that drops the evidence-only instruction and the length limit (android and webmasters). With the open-weight generator, question and generated answer entries are safe in 2 of 3 android configurations, and served answers from depth-2 entries state the root solution in 46% against 52% from threads. With the

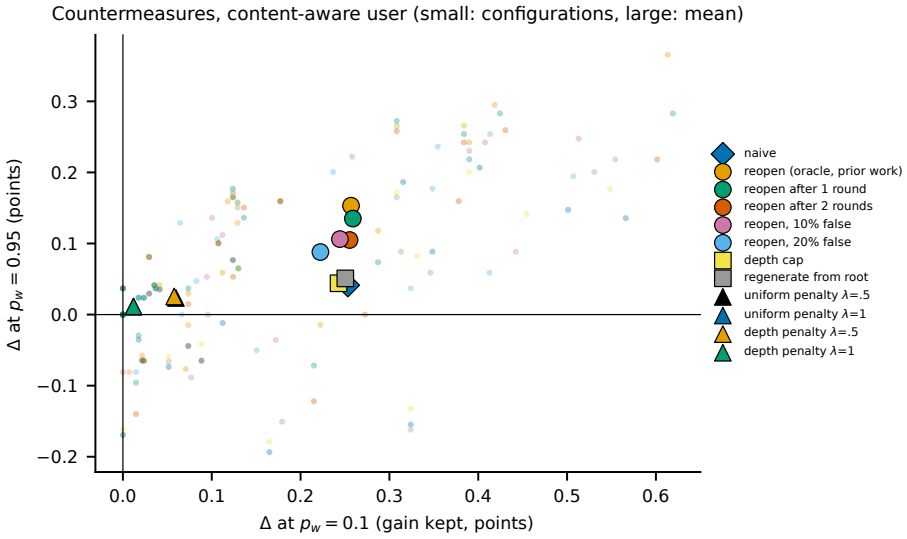


Figure 7: Gain kept at $p_w = 0.1$ against the outcome at $p_w = 0.95$, per configuration (small) and mean (large), content-aware user.

open prompt, the entries are safe in 2 of 6 configurations, the mean Δ is +0.23 points at $p_w = 0.1$ and -0.05 at $p_w = 0.95$, and the share stating the root solution is 83% from threads and 47% from depth-2 entries. Without the evidence-only instruction the answers drift much faster along chains, and on average answers served from entries of every depth now cost correct answers rather than add them. The containment we report in Section 4.3 therefore depends on keeping the generator close to its evidence. The open-weight generator states the root solution less often at every depth and reproduces the main pattern otherwise.

Largest family. Removing the largest family leaves every interval in Table 1 on the same side of zero.

5 Discussion and limitations

What a practitioner should take from this. Writing accepted generated answers back is a small lever in both directions on forums like these: most questions cannot benefit, and those that can are fixed only a few percent of the time. The risk worth managing is the first-level one, a wrongly accepted answer retrieved for the next similar question, not a runaway chain. Store the question with the answer, since that removes most of the retrievers' preference for model-written text. Instruct the generator to answer from the retrieved evidence: without that instruction, the share of answers that keep the solution falls about twice as steeply along chains and stored entries begin to cost correct answers. Let users reopen answers that did not work and remove the corresponding entries; this helps even when reopening is late or sometimes wrong. Rules that act on provenance depth, a depth cap, regeneration from the root thread or depth-scaled down-weighting, do not improve on this, and down-weighting buys safety only by switching the loop off.

Limitations. Acceptance is simulated with a constant p_w, while real users probably accept fluent wrong answers more often than hesitant ones; the content-aware user captures part of this, not all. The content label comes from one judge model. It agrees with two others at the level they agree with each other, but no answers were labelled by people, and judges can favour model-written text (Panickssery et al., 2024). Correctness is anchored on the root thread's human answer, which ignores answers that are right from the model's own knowledge despite wrong evidence; this matters more for the open prompt. The streams are one pass through the held-out questions of each forum, between 1,358 and 1,696, and the share of answers served from deep entries is still growing at the end, so a long-lived system could see more compounding than we can observe; our harm-by-depth numbers bound what happens within this horizon, not beyond it. The

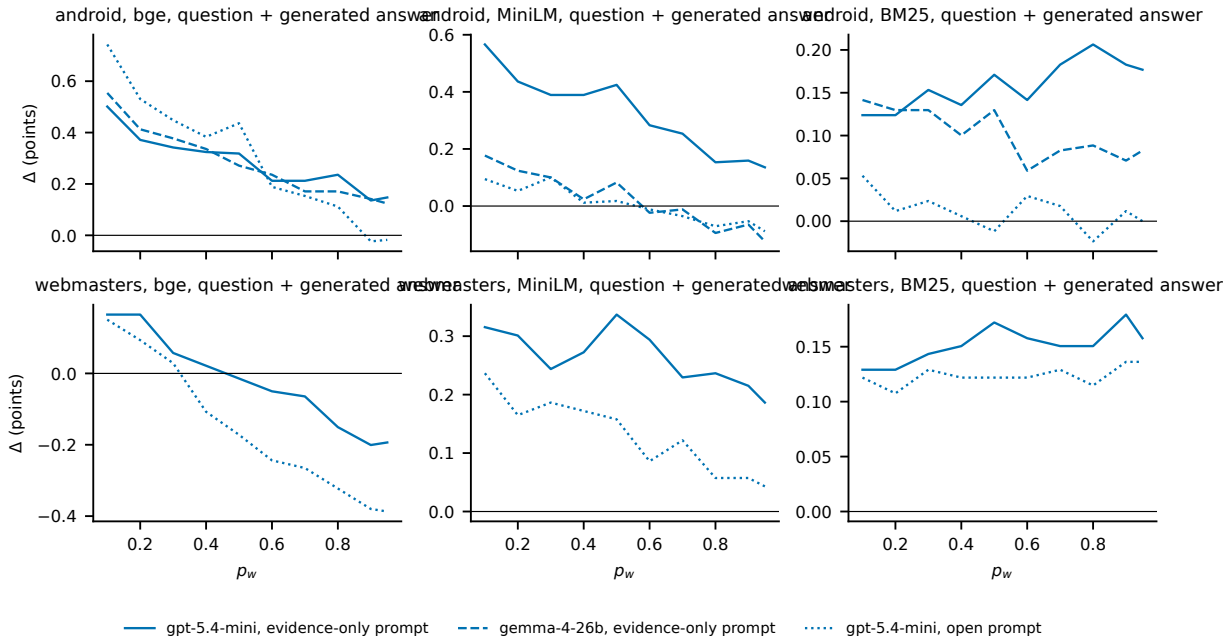


Figure 8: Δ for question and generated answer entries, content-aware user, with the main generator and prompt, an open-weight generator, and an open prompt without the evidence-only instruction or length limit.

questions are CQADupStack titles, the four forums are technical, and the static accuracy of the retrievers is low, which limits how much any entry can help. Answers were fetched in 2026 for a 2014 snapshot, so some accepted answers have been edited since.

6 Conclusion

When generated answers become evidence in a human-gated support loop, the text does drift along provenance chains and retrievers do prefer model-written entries, but within the horizon of real repeat questions the harm stays at the first level and is small against what write-back could achieve. The most useful protections are the simplest ones: keep the generator close to its evidence, store the question with the answer, and let users reopen what did not work.

Broader Impact Statement

This work studies a design that support and service desks already deploy: storing answers that users accepted so that later questions can be answered from them. The results are meant to help practitioners decide whether and how to do this safely. The data are public community question-and-answer threads, used under their CC BY-SA licence, and no personal or customer data were used. The simulated user is a simplification, and a deployment that relies on our numbers without measuring its own acceptance of wrong answers could still store harmful content. Generated answers are stored only after acceptance in our setting; systems that write back without any human or automated check are outside what we measure.

References

Sina Alemohammad, Josue Casco-Rodriguez, Lorenzo Luzi, Ahmed Imtiaz Humayun, Hossein Babaei, Daniel LeJeune, Ali Siahkoohi, and Richard Baraniuk. Self-consuming generative models go MAD. In *The Twelfth International Conference on Learning Repre-*

sentations (ICLR), 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/ebc042e767de551803ccfcc45e2454f5-Abstract-Conference.html. arXiv:2307.01850.

Allison J. B. Chaney, Brandon M. Stewart, and Barbara E. Engelhardt. How algorithmic confounding in recommendation systems increases homogeneity and decreases utility. In *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys '18)*, pp. 224–232, Vancouver, BC, Canada, 2018. ACM. doi: 10.1145/3240323.3240370.

Qinwen Chen, Wenbiao Tao, Zhiwei Zhu, Mingfan Xi, Liangzhong Guo, Yuan Wang, Wei Wang, and Yunshi Lan. ComRAG: Retrieval-augmented generation with dynamic vector stores for real-time community question answering in industry. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pp. 749–763, Vienna, Austria, July 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.acl-industry.53. URL https://aclanthology.org/2025.acl-industry.53/.

Wenhui Chen, Jianlin Chen, Ziyao Lin, and Chi Man Vong. Self-cleaning and captured anyway: One measured primitive for error in a store an agent writes to itself, and what a falling score actually measures, 2026. URL https://arxiv.org/abs/2609.25052.

Xiaoyang Chen, Ben He, Hongyu Lin, Xianpei Han, Tianshu Wang, Boxi Cao, Le Sun, and Yingfei Sun. Spiral of silence: How is large language model killing information retrieval?—A case study on open domain question answering. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14930–14951, Bangkok, Thailand, August 2024a. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.798. URL https://aclanthology.org/2024.acl-long.798/.

Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In *Advances in Neural Information Processing Systems*, volume 37, pp. 130185–130213, 2024b. doi: 10.52202/079017-4136. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb113910e9c3f6242541c1652e30dfd6-Abstract-Conference.html. arXiv:2407.12784.

Teja Chinthala. Bidirectional RAG: Safe self-improving retrieval-augmented generation through multi-stage validation, 2025. URL https://arxiv.org/abs/2512.22199.

Vishisht Choudhary. Keep your receipts: Trustworthy cleanup of agent-built knowledge graphs, 2026. URL https://arxiv.org/abs/2610.17719. NeurIPS 2026 Workshop: Who Verifies the Agents? Toward Reliable Agent Development.

Consortium for Service Innovation. KCS v6 practices guide. https://library.serviceinnovation.org/KCS/KCS_v6/KCS_v6_Practices_Guide, 2016. Knowledge-Centered Service (KCS) v6, released 21 April 2016.

Sunhao Dai, Weihao Liu, Yuqi Zhou, Liang Pang, Rongju Ruan, Gang Wang, Zhenhua Dong, Jun Xu, and Ji-Rong Wen. Cocktail: A comprehensive information retrieval benchmark with LLM-generated documents integration. In *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 7052–7074, Bangkok, Thailand, 2024a. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.421.

Sunhao Dai, Yuqi Zhou, Liang Pang, Weihao Liu, Xiaolin Hu, Yong Liu, Xiao Zhang, Gang Wang, and Jun Xu. Neural retrievers are biased towards LLM-generated content. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, pp. 526–537, Barcelona, Spain, 2024b. ACM. doi: 10.1145/3637528.3671882. arXiv:2310.20501.

R. Maria del Rio-Chanona, Nadzeya Laurentsyeva, and Johannes Wachs. Large language models reduce public knowledge sharing on online Q&A platforms. *PNAS Nexus*, 3(9):pgae400, 2024. doi: 10.1093/pnasnexus/pgae400.

Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. Memory injection attacks on LLM agents via query-only interaction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL https://arxiv.org/abs/2503.03704. arXiv v1/v2 titled "A Practical Memory Injection Attack against LLM Agents".

Gregory Druck and Ethan Smith. RAG collapse: LLM responses collapse when retrieved documents are self-authored, 2026. URL https://arxiv.org/abs/2608.22118.

Damien Ferbach, Quentin Bertrand, Avishek Joey Bose, and Gauthier Gidel. Self-consuming generative models with curated data provably optimize human preferences. In *Advances in Neural Information Processing Systems*, volume 37, pp. 102531–102567, 2024. doi: 10.52202/079017-3256. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/b9e88ae0308cf82d0b0f634ddbdf809a-Abstract-Conference.html. arXiv:2407.09499.

Michele Garetto, Alessandro Cornacchia, Franco Galante, Emilio Leonardi, Alessandro Nordio, and Alberto Tarable. Information retrieval in the age of generative AI: The RGB model. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '25, pp. 602–612, Padua, Italy, 2025. ACM. doi: 10.1145/3726302.3730008. arXiv:2504.20610.

Matthias Gerstgrasser, Rylan Schaeffer, Apratim Dey, Rafael Rafailov, Henry Sleight, John Hughes, Tomasz Korbak, Rajashree Agrawal, Dhruv Pai, Andrey Gromov, Daniel A. Roberts, Diyi Yang, David L. Donoho, and Sanmi Koyejo. Is model collapse inevitable? breaking the curse of recursion by accumulating real and synthetic data, 2024. URL https://arxiv.org/abs/2404.01413.

Pavan Kumar Guthikonda. When accepted answers become evidence: Feedback loops in self-updating retrieval. Zenodo preprint, version 3, 2026. All versions: doi:10.5281/zenodo.23089817.

Doris Hoogeveen, Karin M. Verspoor, and Timothy Baldwin. CQADupStack: A benchmark data set for community question-answering research. In *Proceedings of the 20th Australasian Document Computing Symposium (ADCS)*, pp. 3:1–3:8. ACM, 2015.

Wei Huang, Keping Bi, Yinqiong Cai, Wei Chen, Jiafeng Guo, and Xueqi Cheng. How do LLM-generated texts impact term-based retrieval models?, 2025.

Wei Huang, Keping Bi, Yinqiong Cai, Wei Chen, Jiafeng Guo, and Xueqi Cheng. Relevance supervision can teach neural retrievers to prefer LLM-generated text, 2026. v1 (Apr 2026) titled "Data, Not Model: Explaining Bias toward LLM Texts in Neural Retrievers".

Ray Jiang, Silvia Chiappa, Tor Lattimore, András György, and Pushmeet Kohli. Degenerate feedback loops in recommender systems. In *Proceedings of the 2019 AAAI/ACM Conference on AI, Ethics, and Society (AIES '19)*, pp. 383–390. ACM, 2019. doi: 10.1145/3306618.3314288.

Arulnidhi Karunanidhi. Utility under attack: Agent memory poisoning and the limits of content screening and provenance ranking, 2026. URL https://arxiv.org/abs/2608.21230.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33*, 2020.

Yufeng Li, Sheng Yao, Peng Zeng, Dezhang Kong, Shi Lin, Changting Lin, Xun Wang, and Meng Han. Memory echo in single-agent long-term memory: Lifecycle evaluation and provenance-guided mitigation, 2026. URL https://arxiv.org/abs/2610.17763.

Fengrui Liu, Xiao He, and Tieying Zhang. Help without being asked: A deployed proactive agent system for on-call support with continuous self-improvement. arXiv preprint arXiv:2604.09579, 2026.

Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, volume 37, pp. 68772–68802, 2024. doi: 10.52202/079017-2197. arXiv:2404.13076.

Juan Perdomo, Tijana Zrnic, Celestine Mendler-Dünner, and Moritz Hardt. Performative prediction. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pp. 7599–7609. PMLR, 2020. URL https://proceedings.mlr.press/v119/perdomo20a.html.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of EMNLP-IJCNLP 2019*, pp. 3982–3992, 2019.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Ilia Shumailov, Zakhar Shumaylov, Yiren Zhao, Nicolas Papernot, Ross Anderson, and Yarin Gal. AI models collapse when trained on recursively generated data. *Nature*, 631(8022):755–759, 2024. doi: 10.1038/s41586-024-07566-y.

Hexiang Tan, Fei Sun, Wanli Yang, Yuanzhuo Wang, Qi Cao, and Xueqi Cheng. Blinded by generated contexts: How language models merge generated and retrieved contexts when knowledge conflicts? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6207–6227, Bangkok, Thailand, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.337. URL https://aclanthology.org/2024.acl-long.337/. arXiv:2401.11911.

Rohan Taori and Tatsunori Hashimoto. Data feedback loops: Model-driven amplification of dataset biases. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, pp. 33883–33920. PMLR, 2023. URL https://proceedings.mlr.press/v202/taori23a.html.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2021.

Chih Hui Wang, Mengdie Tu, Qianyun Zhang, Wei Wu, Lili Zhou, Mingqi Shen, and Changshuai Wei. Self-evolving agentic customer support system at LinkedIn, 2026a.

Haoyu Wang, Sunhao Dai, Haiyuan Zhao, Liang Pang, Xiao Zhang, Gang Wang, Zhenhua Dong, Jun Xu, and Ji-Rong Wen. Perplexity trap: PLM-based retrievers overrate low perplexity documents. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=U1T6sq12uj. arXiv:2503.08684.

Ruhan Wang, Kishan Panaganti, and Dongruo Zhou. More memory, worse agents: Error reproduction and anti-persistence in LLM agents, 2026b. URL https://arxiv.org/abs/2610.27406.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33*, 2020.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024. doi: 10.1145/3626772.3657878.

William Xion and Wolfgang Nejdl. Training-induced bias toward LLM-generated content in dense retrieval. In *Advances in Information Retrieval (ECIR 2026)*, Lecture Notes in Computer Science, pp. 83–97. Springer Nature Switzerland, 2026. doi: 10.1007/978-3-032-21289-4_6. arXiv:2602.10833; LNCS volume number not confirmed.

Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 623–645, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.27. URL https://aclanthology.org/2026.acl-long.27/. arXiv:2505.16067.

Hongyeon Yu, Dongchan Kim, and Young-Bum Kim. Retrieval collapses when AI pollutes the web. In *Proceedings of the ACM Web Conference 2026*, WWW '26, pp. 8745–8748. ACM, 2026. doi: 10.1145/3774904.3792955. arXiv:2602.16136.

Muru Zhang, Ofir Press, William Merrill, Alisa Liu, and Noah A. Smith. How language model hallucinations can snowball. In Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (eds.), *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 59670–59684. PMLR, 2024. URL `https://proceedings.mlr.press/v235/zhang24ay.html`. arXiv:2305.13534 (2023).

Yan Zhang and Shibo Li. ConsistencyGate: Preventing memory contamination in LLM agents via self-consistency admission control, 2026. URL `https://arxiv.org/abs/2607.22962`.

Yuqi Zhou, Sunhao Dai, Liang Pang, Gang Wang, Zhenhua Dong, Jun Xu, and Ji-Rong Wen. Exploring the escalation of source bias in user, data, and recommender system feedback loop. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '25, pp. 1676–1686, Padua, Italy, 2025. ACM. doi: 10.1145/3726302.3729972. arXiv:2405.17998; v1 (May 2024) titled "Source Echo Chamber: Exploring the Escalation of Source Bias in User, Data, and Recommender System Feedback Loop".

Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. PoisonedRAG: Knowledge corruption attacks to Retrieval-Augmented generation of large language models. In *34th USENIX Security Symposium (USENIX Security 25)*, pp. 3827–3844, Seattle, WA, August 2025. USENIX Association. ISBN 978-1-939133-52-6. URL `https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag`.

A Deviations from the initial plan

The study plan was written before the first simulation and is included in the supplementary material. Table 5 lists every departure from it. The most important ones followed an internal review of the first version of the code and the paper: the first version labelled correctness only by the root thread, so the generated text could not create an error; its source-bias test did not hold content fixed; its depth-aware estimate was, by construction, the first-order estimate divided by a constant; and its reopen rule was an oracle. All four are corrected here and the first-version numbers are not used.

B Prompts

Generator, main prompt. *You are a technical support assistant. Answer the user's question using only the evidence provided. If the evidence does not fully answer the question, say what it does cover and give the closest guidance it supports. At most 120 words. Plain text, no headings, no mention of the evidence or of these instructions.* The user message is the question followed by the evidence. A thread is shown as its text followed by its accepted or top-voted answer, or a note that it has none. A stored entry is shown as "Previously resolved case" with its question and stored answer.

Generator, open prompt. *You are a technical support assistant. Answer the user's question as helpfully as you can. A previously resolved case or thread is provided as context; use it where it helps. Plain text.* No length limit beyond the token cap.

Judge. *You grade answers from a technical support system. You are given the user's question, a reference answer written by the community and accepted as the solution, and the system's answer. Reply with exactly one word. YES: the system's answer gives the same solution as the reference, possibly with less detail. PARTLY: it overlaps with the reference but misses or changes the key step. NO: it gives a different solution, a wrong solution, or no usable solution. ABSTAIN: it says the information available does not answer the question and does not offer a solution.* The reference is the root thread's human answer, cut to 600 words.

Table 5: Departures from the initial plan.

Item	Planned	Done, and why
H1	Compare crossover and capture of generated entries with the pointer entries of the earlier paper.	Kept, and added a same-content control with length-matched human text, because the planned comparison confounds authorship with content and length.
H2	Compounding shown by deep entries growing with rounds and by drift.	Kept, and added content labels for every served answer and counterfactual harm by served depth, because growth alone does not show harm.
H3	Depth-aware plug-in estimate, tested out of sample.	The first version used an estimator that only rescaled the first-order one. Replaced by the estimator of Section 4.4, tested out of sample as planned.
H4	Rank reopen, cited source and down-weighting by loss removed per gain lost.	Reopen in the plan was the oracle rule of the earlier paper. Added delayed and noisy reopen, a depth cap, regeneration from the root and depth-proportional down-weighting. Storing the cited source is reported under H1, where it is an entry design.
Correctness	Evidence-based root label, plus a 200-answer manual audit.	Root label kept as one of two user models. The content label from an independent judge replaces the manual audit as the main label, validated against two further judges on 900 answers (Appendix C). No answers were judged by hand.
Data	Four CQADupStack forums and MS MARCO query families.	MS MARCO dropped: its passages have no community answer to judge content against.
Generators	One generator, one open-weight replication.	Kept, and added a second prompt without the evidence-only instruction or length limit on two forums (Section 4.6).

C Judge validation and faithfulness audit

Table 6: Faithfulness audit of 900 cached answers, 25 per forum at each evidence depth and order of sampling, judged by three models against the root thread's human answer (ABSTAIN counted as NO). Cohen's κ: DeepSeek and gpt-5.4-mini 0.64, DeepSeek and gemma 0.48, gpt-5.4-mini and gemma 0.52. Trend p: Cochran–Armitage test for the share of NO across depths.

	DeepSeek-V4.1-Flash			gpt-5.4-mini			gemma-4-26b		
evidence depth	yes	partly	no	yes	partly	no	yes	partly	no
0 (thread)	73%	20%	7%	73%	17%	10%	86%	3%	12%
1	61%	33%	7%	61%	28%	11%	78%	7%	15%
2 or more	55%	31%	14%	57%	27%	16%	73%	8%	19%
trend p (no)			0.003			0.025			0.009

D Full results

Table 7: Entry designs by user model and configuration: crossover p^* with bootstrap interval, Δ (points, mean $\pm$ 95% interval over orders) at $p_w = 0.1$ and 0.95, share of answers served from entries and steal rate at $p_w = 0.95$. Pointer rows are from the earlier paper (root label).

config	user	design	p^*	[95% interval]	Δ at 0.1	Δ at 0.95	self	steal
android bge	root	question only (pointer)	0.47	[0.37, 0.54]	+2.16 ± 0.72	-1.87 ± 0.65	65%	33.5%
android bge	root	question + thread (pointer)	0.77	[0.49, 0.95]	+0.27 ± 0.07	-0.04 ± 0.14	19%	3.5%
android bge	root	answer	0.74	[0.57, 0.83]	+0.77 ± 0.12	-0.19 ± 0.21	27%	9.6%
android bge	root	question + answer	>0.95	[0.83, 0.95]	+0.64 ± 0.15	+0.14 ± 0.24	19%	5.8%
android bge	root	answer + thread	0.89	[0.81, 0.95]	+0.81 ± 0.12	-0.04 ± 0.21	25%	8.7%
android bge	root	human answer (ctrl)	0.60	[0.34, 0.74]	+0.31 ± 0.07	-0.16 ± 0.13	19%	5.6%
android bge	root	question + human answer (ctrl)	>0.95	[0.85, 0.95]	+0.37 ± 0.10	+0.18 ± 0.16	15%	3.5%
android bge	content	answer	0.60	[0.46, 0.83]	+0.51 ± 0.13	-0.18 ± 0.23	28%	9.6%
android bge	content	question + answer	>0.95	[0.84, 0.95]	+0.50 ± 0.15	+0.15 ± 0.24	19%	5.8%
android bge	content	answer + thread	0.88	[0.60, 0.95]	+0.61 ± 0.14	-0.05 ± 0.19	25%	8.7%
android bge	content	human answer (ctrl)	0.60	[0.29, 0.75]	+0.27 ± 0.09	-0.14 ± 0.11	19%	5.6%
android bge	content	question + human answer (ctrl)	>0.95	[0.74, 0.95]	+0.28 ± 0.09	+0.10 ± 0.14	15%	3.5%
android MiniLM	root	question only (pointer)	0.67	[0.55, 0.78]	+0.83 ± 0.31	-0.29 ± 0.24	34%	9.6%
android MiniLM	root	question + thread (pointer)	>0.95	[0.95, 0.95]	+0.52 ± 0.19	+0.25 ± 0.14	24%	3.0%
android MiniLM	root	answer	>0.95	[0.95, 0.95]	+0.33 ± 0.10	+0.15 ± 0.12	10%	2.4%
android MiniLM	root	question + answer	>0.95	[0.95, 0.95]	+0.64 ± 0.14	+0.25 ± 0.17	15%	2.8%
android MiniLM	root	answer + thread	0.38	[0.20, 0.75]	+0.12 ± 0.06	-0.14 ± 0.14	9%	2.6%
android MiniLM	root	human answer (ctrl)	>0.95	[0.19, 0.95]	+0.11 ± 0.07	+0.04 ± 0.08	8%	1.0%
android MiniLM	root	question + human answer (ctrl)	>0.95	[0.84, 0.95]	+0.28 ± 0.07	+0.08 ± 0.14	16%	2.8%
android MiniLM	content	answer	>0.95	[0.95, 0.95]	+0.27 ± 0.09	+0.12 ± 0.11	10%	2.4%
android MiniLM	content	question + answer	>0.95	[0.87, 0.95]	+0.57 ± 0.13	+0.14 ± 0.18	15%	2.8%
android MiniLM	content	answer + thread	0.15	[0.10, 0.22]	+0.04 ± 0.08	-0.22 ± 0.11	9%	2.6%
android MiniLM	content	human answer (ctrl)	0.75	[0.10, 0.95]	+0.05 ± 0.07	+0.01 ± 0.07	8%	1.0%
android MiniLM	content	question + human answer (ctrl)	>0.95	[0.58, 0.95]	+0.22 ± 0.09	+0.01 ± 0.13	16%	2.8%
android BM25	root	question only (pointer)	>0.95	[0.34, 0.95]	+0.08 ± 0.05	+0.03 ± 0.10	7%	1.8%
android BM25	root	question + thread (pointer)	>0.95	[0.46, 0.95]	+0.27 ± 0.18	+0.02 ± 0.19	19%	4.5%
android BM25	root	answer	>0.95	[0.17, 0.95]	+0.05 ± 0.04	+0.05 ± 0.07	6%	1.1%
android BM25	root	question + answer	>0.95	[0.95, 0.95]	+0.15 ± 0.07	+0.20 ± 0.08	10%	1.2%
android BM25	root	answer + thread	>0.95	[0.95, 0.95]	+0.24 ± 0.08	+0.29 ± 0.11	9%	0.3%
android BM25	root	human answer (ctrl)	<0.10	[0.10, 0.10]	-0.04 ± 0.02	-0.01 ± 0.03	3%	0.5%
android BM25	root	question + human answer (ctrl)	>0.95	[0.95, 0.95]	+0.22 ± 0.08	+0.27 ± 0.09	10%	1.4%
android BM25	content	answer	>0.95	[0.13, 0.95]	+0.02 ± 0.03	+0.03 ± 0.06	6%	1.1%
android BM25	content	question + answer	>0.95	[0.95, 0.95]	+0.12 ± 0.07	+0.18 ± 0.07	10%	1.2%
android BM25	content	answer + thread	>0.95	[0.95, 0.95]	+0.22 ± 0.07	+0.28 ± 0.08	9%	0.3%
android BM25	content	human answer (ctrl)	<0.10	[0.10, 0.10]	-0.04 ± 0.02	-0.01 ± 0.03	3%	0.5%
android BM25	content	question + human answer (ctrl)	>0.95	[0.95, 0.95]	+0.20 ± 0.07	+0.24 ± 0.09	10%	1.4%
unix bge	root	question only (pointer)	0.26	[0.24, 0.29]	+1.03 ± 0.38	-2.68 ± 0.57	56%	29.9%
unix bge	root	question + thread (pointer)	0.81	[0.58, 0.95]	+0.26 ± 0.08	-0.04 ± 0.14	11%	2.7%
unix bge	root	answer	0.83	[0.61, 0.95]	+0.45 ± 0.13	-0.08 ± 0.20	16%	5.7%
unix bge	root	question + answer	>0.95	[0.95, 0.95]	+0.44 ± 0.08	+0.25 ± 0.15	8%	2.0%
unix bge	root	answer + thread	>0.95	[0.95, 0.95]	+0.48 ± 0.07	+0.22 ± 0.13	12%	3.2%
unix bge	root	human answer (ctrl)	<0.10	[0.10, 0.10]	-0.04 ± 0.05	-0.40 ± 0.10	13%	4.2%
unix bge	root	question + human answer (ctrl)	>0.95	[0.95, 0.95]	+0.38 ± 0.07	+0.12 ± 0.12	10%	3.0%
unix bge	content	answer	0.67	[0.53, 0.95]	+0.40 ± 0.14	-0.16 ± 0.20	16%	5.7%
unix bge	content	question + answer	>0.95	[0.95, 0.95]	+0.40 ± 0.10	+0.21 ± 0.14	8%	2.0%
unix bge	content	answer + thread	>0.95	[0.88, 0.95]	+0.43 ± 0.09	+0.14 ± 0.11	12%	3.2%
unix bge	content	human answer (ctrl)	<0.10	[0.10, 0.10]	-0.04 ± 0.05	-0.40 ± 0.10	13%	4.2%
unix bge	content	question + human answer (ctrl)	>0.95	[0.95, 0.95]	+0.38 ± 0.07	+0.12 ± 0.12	10%	3.0%
unix MiniLM	root	question only (pointer)	0.63	[0.52, 0.74]	+0.80 ± 0.17	-0.47 ± 0.24	27%	11.2%
unix MiniLM	root	question + thread (pointer)	>0.95	[0.84, 0.95]	+0.41 ± 0.14	+0.10 ± 0.16	16%	3.0%
unix MiniLM	root	answer	0.84	[0.59, 0.95]	+0.27 ± 0.08	-0.05 ± 0.12	8%	3.2%
unix MiniLM	root	question + answer	>0.95	[0.95, 0.95]	+0.35 ± 0.10	+0.11 ± 0.13	10%	3.2%
unix MiniLM	root	answer + thread	>0.95	[0.95, 0.95]	+0.37 ± 0.08	+0.32 ± 0.08	6%	1.2%
unix MiniLM	root	human answer (ctrl)	>0.95	[0.64, 0.95]	+0.13 ± 0.08	+0.02 ± 0.13	4%	1.5%
unix MiniLM	root	question + human answer (ctrl)	>0.95	[0.95, 0.95]	+0.41 ± 0.12	+0.16 ± 0.14	9%	3.1%
unix MiniLM	content	answer	0.78	[0.56, 0.95]	+0.27 ± 0.08	-0.08 ± 0.13	8%	3.2%
unix MiniLM	content	question + answer	>0.95	[0.92, 0.95]	+0.35 ± 0.10	+0.09 ± 0.14	10%	3.2%
unix MiniLM	content	answer + thread	>0.95	[0.95, 0.95]	+0.37 ± 0.08	+0.28 ± 0.08	6%	1.2%
unix MiniLM	content	human answer (ctrl)	0.81	[0.52, 0.95]	+0.13 ± 0.08	-0.05 ± 0.13	4%	1.5%
unix MiniLM	content	question + human answer (ctrl)	>0.95	[0.86, 0.95]	+0.37 ± 0.13	+0.11 ± 0.13	9%	3.1%
unix BM25	root	question only (pointer)	>0.95	[0.55, 0.95]	+0.06 ± 0.04	+0.04 ± 0.05	2%	0.4%
unix BM25	root	question + thread (pointer)	>0.95	[0.95, 0.95]	+0.12 ± 0.05	+0.09 ± 0.05	9%	0.4%
unix BM25	root	answer	0.40	[0.21, 0.95]	+0.07 ± 0.07	-0.03 ± 0.07	5%	2.4%
unix BM25	root	question + answer	>0.95	[0.39, 0.95]	+0.14 ± 0.09	+0.01 ± 0.07	7%	2.8%
unix BM25	root	answer + thread	>0.95	[0.95, 0.95]	+0.17 ± 0.06	+0.13 ± 0.04	7%	1.3%
unix BM25	root	human answer (ctrl)	<0.10	[0.10, 0.30]	-0.01 ± 0.01	-0.12 ± 0.04	2%	1.3%
unix BM25	root	question + human answer (ctrl)	0.68	[0.37, 0.94]	+0.08 ± 0.05	-0.04 ± 0.04	4%	1.4%
unix BM25	content	answer	0.35	[0.13, 0.93]	+0.05 ± 0.06	-0.05 ± 0.07	5%	2.4%
unix BM25	content	question + answer	0.89	[0.35, 0.95]	+0.11 ± 0.08	-0.01 ± 0.06	7%	2.8%
unix BM25	content	answer + thread	>0.95	[0.95, 0.95]	+0.14 ± 0.05	+0.11 ± 0.04	7%	1.3%
unix BM25	content	human answer (ctrl)	<0.10	[0.10, 0.30]	-0.01 ± 0.01	-0.12 ± 0.04	2%	1.3%
unix BM25	content	question + human answer (ctrl)	0.68	[0.37, 0.93]	+0.08 ± 0.05	-0.04 ± 0.04	4%	1.4%
webmasters bge	root	question only (pointer)	0.14	[0.12, 0.19]	+0.19 ± 0.16	-1.79 ± 0.28	72%	31.3%
webmasters bge	root	question + thread (pointer)	0.32	[0.14, 0.66]	+0.07 ± 0.07	-0.12 ± 0.09	24%	3.8%
webmasters bge	root	answer	0.36	[0.24, 0.46]	+0.19 ± 0.11	-0.36 ± 0.14	31%	8.1%
webmasters bge	root	question + answer	0.58	[0.36, 0.77]	+0.18 ± 0.07	-0.15 ± 0.09	18%	5.0%
webmasters bge	root	answer + thread	0.40	[0.27, 0.53]	+0.14 ± 0.09	-0.32 ± 0.11	29%	9.1%
webmasters bge	root	human answer (ctrl)	>0.95	[0.38, 0.95]	+0.32 ± 0.20	+0.06 ± 0.20	22%	4.8%
webmasters bge	root	question + human answer (ctrl)	0.30	[0.10, 0.84]	+0.07 ± 0.19	-0.19 ± 0.10	19%	4.4%
webmasters bge	content	answer	0.34	[0.20, 0.45]	+0.16 ± 0.11	-0.38 ± 0.12	31%	8.1%
webmasters bge	content	question + answer	0.46	[0.30, 0.70]	+0.16 ± 0.07	-0.19 ± 0.08	18%	5.0%
webmasters bge	content	answer + thread	0.34	[0.25, 0.46]	+0.11 ± 0.09	-0.39 ± 0.11	29%	9.1%
webmasters bge	content	human answer (ctrl)	0.79	[0.36, 0.95]	+0.31 ± 0.19	-0.05 ± 0.16	22%	4.8%
webmasters bge	content	question + human answer (ctrl)	0.29	[0.10, 0.82]	+0.06 ± 0.19	-0.22 ± 0.09	19%	4.4%
webmasters MiniLM	root	question only (pointer)	0.77	[0.56, 0.92]	+0.98 ± 0.32	-0.32 ± 0.26	42%	14.2%
webmasters MiniLM	root	question + thread (pointer)	0.90	[0.26, 0.95]	+0.22 ± 0.09	-0.04 ± 0.14	28%	5.8%
webmasters MiniLM	root	answer	>0.95	[0.95, 0.95]	+0.29 ± 0.14	+0.25 ± 0.14	11%	1.9%
webmasters MiniLM	root	question + answer	>0.95	[0.95, 0.95]	+0.34 ± 0.15	+0.16 ± 0.19	17%	3.4%
webmasters MiniLM	root	answer + thread	>0.95	[0.95, 0.95]	+0.24 ± 0.09	+0.22 ± 0.11	10%	1.0%
webmasters MiniLM	root	human answer (ctrl)	>0.95	[0.95, 0.95]	+0.18 ± 0.07	+0.13 ± 0.09	7%	1.3%
webmasters MiniLM	root	question + human answer (ctrl)	>0.95	[0.70, 0.95]	+0.11 ± 0.07	+0.09 ± 0.12	18%	2.0%
webmasters MiniLM	content	answer	>0.95	[0.95, 0.95]	+0.29 ± 0.14	+0.24 ± 0.13	11%	1.9%
webmasters MiniLM	content	question + answer	>0.95	[0.95, 0.95]	+0.32 ± 0.15	+0.19 ± 0.17	17%	3.4%
webmasters MiniLM	content	answer + thread	>0.95	[0.95, 0.95]	+0.22 ± 0.09	+0.22 ± 0.09	10%	1.0%
webmasters MiniLM	content	human answer (ctrl)	>0.95	[0.95, 0.95]	+0.18 ± 0.07	+0.12 ± 0.10	7%	1.3%
webmasters MiniLM	content	question + human answer (ctrl)	>0.95	[0.93, 0.95]	+0.11 ± 0.07	+0.09 ± 0.12	18%	2.0%
webmasters BM25	root	question only (pointer)	<0.10	[0.10, 0.17]	-0.01 ± 0.03	-0.08 ± 0.03	7%	0.9%
webmasters BM25	root	question + thread (pointer)	<0.10	[0.10, 0.10]	-0.08 ± 0.06	-0.27 ± 0.07	17%	4.5%
webmasters BM25	root	answer	>0.95	[0.95, 0.95]	+0.09 ± 0.03	+0.11 ± 0.06	10%	1.5%
webmasters BM25	root	question + answer	>0.95	[0.95, 0.95]	+0.14 ± 0.05	+0.16 ± 0.07	14%	1.8%
webmasters BM25	root	answer + thread	>0.95	[0.46, 0.95]	+0.09 ± 0.02	+0.09 ± 0.09	12%	1.7%
webmasters BM25	root	human answer (ctrl)	0.16	[0.10, 0.40]	+0.03 ± 0.04	-0.04 ± 0.06	5%	1.5%
webmasters BM25	root	question + human answer (ctrl)	>0.95	[0.17, 0.95]	+0.09 ± 0.07	+0.07 ± 0.09	9%	1.5%
webmasters BM25	content	answer	>0.95	[0.95, 0.95]	+0.07 ± 0.03	+0.11 ± 0.05	10%	1.5%
webmasters BM25	content	question + answer	>0.95	[0.95, 0.95]	+0.13 ± 0.06	+0.16 ± 0.08	14%	1.8%
webmasters BM25	content	answer + thread	0.80	[0.40, 0.95]	+0.08 ± 0.03	+0.01 ± 0.09	12%	1.7%
webmasters BM25	content	human answer (ctrl)	0.12	[0.10, 0.20]	+0.01 ± 0.03	-0.04 ± 0.05	5%	1.5%
webmasters BM25	content	question + human answer (ctrl)	0.47	[0.12, 0.95]	+0.05 ± 0.05	+0.01 ± 0.07	9%	1.5%
mathematica bge	root	question only (pointer)	<0.10	[0.10, 0.11]	-0.20 ± 0.30	-2.58 ± 0.30	65%	39.0%
mathematica bge	root	question + thread (pointer)	0.70	[0.36, 0.95]	+0.11 ± 0.03	-0.02 ± 0.06	11%	2.5%
mathematica bge	root	answer	0.29	[0.23, 0.52]	+0.18 ± 0.10	-0.46 ± 0.20	22%	11.0%
mathematica bge	root	question + answer	<0.10	[0.10, 0.30]	-0.00 ± 0.09	-0.14 ± 0.06	10%	2.9%
mathematica bge	root	answer + thread	0.64	[0.27, 0.79]	+0.15 ± 0.07	-0.07 ± 0.12	17%	4.5%
mathematica bge	root	human answer (ctrl)	0.11	[0.10, 0.20]	+0.01 ± 0.12	-0.71 ± 0.17	22%	13.7%
mathematica bge	root	question + human answer (ctrl)	0.22	[0.10, 0.45]	+0.04 ± 0.08	-0.32 ± 0.10	14%	7.5%
mathematica bge	content	answer	0.25	[0.18, 0.44]	+0.13 ± 0.10	-0.54 ± 0.19	22%	11.0%
mathematica bge	content	question + answer	<0.10	[0.10, 0.28]	-0.00 ± 0.09	-0.17 ± 0.06	10%	2.9%
mathematica bge	content	answer + thread	0.60	[0.27, 0.76]	+0.14 ± 0.07	-0.10 ± 0.12	17%	4.5%
mathematica bge	content	human answer (ctrl)	0.11	[0.10, 0.18]	+0.01 ± 0.12	-0.74 ± 0.16	22%	13.7%
mathematica bge	content	question + human answer (ctrl)	0.22	[0.10, 0.45]	+0.04 ± 0.08	-0.32 ± 0.10	14%	7.5%
mathematica MiniLM	root	question only (pointer)	0.23	[0.16, 0.31]	+0.33 ± 0.22	-1.15 ± 0.27	46%	22.0%
mathematica MiniLM	root	question + thread (pointer)	<0.10	[0.10, 0.15]	-0.01 ± 0.06	-0.49 ± 0.11	17%	7.2%
mathematica MiniLM	root	answer	>0.95	[0.26, 0.95]	+0.12 ± 0.08	+0.01 ± 0.19	12%	4.4%
mathematica MiniLM	root	question + answer	0.70	[0.28, 0.88]	+0.33 ± 0.11	-0.14 ± 0.21	13%	6.0%
mathematica MiniLM	root	answer + thread	>0.95	[0.95, 0.95]	+0.18 ± 0.08	+0.13 ± 0.11	9%	1.7%
mathematica MiniLM	root	human answer (ctrl)	0.20	[0.10, 0.42]	+0.03 ± 0.06	-0.18 ± 0.07	8%	3.2%
mathematica MiniLM	root	question + human answer (ctrl)	0.68	[0.27, 0.93]	+0.20 ± 0.09	-0.10 ± 0.15	12%	4.7%
mathematica MiniLM	content	answer	0.90	[0.25, 0.95]	+0.11 ± 0.08	-0.01 ± 0.20	12%	4.4%
mathematica MiniLM	content	question + answer	0.65	[0.28, 0.84]	+0.32 ± 0.12	-0.15 ± 0.21	13%	6.0%
mathematica MiniLM	content	answer + thread	>0.95	[0.94, 0.95]	+0.16 ± 0.06	+0.07 ± 0.10	9%	1.7%
mathematica MiniLM	content	human answer (ctrl)	0.20	[0.10, 0.44]	+0.04 ± 0.06	-0.15 ± 0.06	8%	3.2%
mathematica MiniLM	content	question + human answer (ctrl)	0.68	[0.25, 0.94]	+0.20 ± 0.09	-0.10 ± 0.15	12%	4.7%
mathematica BM25	root	question only (pointer)	>0.95	[0.95, 0.95]	+0.00 ± 0.00	+0.00 ± 0.00	3%	0.0%
mathematica BM25	root	question + thread (pointer)	<0.10	[0.10, 0.30]	-0.01 ± 0.03	-0.04 ± 0.04	11%	0.8%
mathematica BM25	root	answer	0.50	[0.10, 0.84]	+0.05 ± 0.07	-0.07 ± 0.11	7%	4.5%
mathematica BM25	root	question + answer	0.52	[0.12, 0.87]	+0.08 ± 0.09	-0.06 ± 0.09	10%	5.0%
mathematica BM25	root	answer + thread	0.50	[0.10, 0.93]	+0.04 ± 0.10	-0.08 ± 0.10	9%	4.2%
mathematica BM25	root	human answer (ctrl)	<0.10	[0.10, 0.10]	-0.06 ± 0.05	-0.04 ± 0.07	3%	2.0%
mathematica BM25	root	question + human answer (ctrl)	>0.95	[0.10, 0.95]	+0.03 ± 0.05	+0.06 ± 0.05	5%	1.4%
mathematica BM25	content	answer	0.35	[0.10, 0.54]	+0.02 ± 0.06	-0.09 ± 0.10	7%	4.5%
mathematica BM25	content	question + answer	0.40	[0.10, 0.60]	+0.05 ± 0.08	-0.07 ± 0.09	10%	5.0%
mathematica BM25	content	answer + thread	0.52	[0.10, 0.90]	+0.04 ± 0.09	-0.06 ± 0.10	9%	4.2%
mathematica BM25	content	human answer (ctrl)	<0.10	[0.10, 0.10]	-0.06 ± 0.05	-0.08 ± 0.05	3%	2.0%
mathematica BM25	content	question + human answer (ctrl)	>0.95	[0.10, 0.95]	+0.03 ± 0.05	+0.01 ± 0.05	5%	1.4%

Table 8: Countermeasures by configuration, content-aware user: Δ (points) at $p_w = 0.1$ and 0.95 and the crossover.

config		*naive write-back*	*reopen, oracle (prior work)*	*reopen after 1 round*	*reopen after 2 rounds*	*reopen after 1 round, 10% false*	*reopen after 1 round, 20% false*	*depth cap (depth 1 only)*	*regenerate from root*	*uniform penalty, $\lambda = 0.5$*	*uniform penalty, $\lambda = 1$*	*depth penalty, $\lambda = 0.5$*	*depth penalty, $\lambda = 1$*
android bge	Δ 0.1	+0.50	+0.42	+0.42	+0.43	+0.44	+0.41	+0.45	+0.53	+0.12	+0.04	+0.11	+0.04
	Δ 0.95	+0.15	+0.29	+0.28	+0.26	+0.09	+0.06	+0.14	+0.20	+0.08	+0.04	+0.06	+0.04
	p^*	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95
android MiniLM	Δ 0.1	+0.57	+0.61	+0.62	+0.60	+0.51	+0.51	+0.55	+0.55	+0.09	+0.02	+0.07	+0.02
	Δ 0.95	+0.14	+0.37	+0.28	+0.22	+0.25	+0.19	+0.18	+0.22	-0.06	-0.04	-0.08	-0.03
	p^*	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	0.29	0.30	0.29	0.35
android BM25	Δ 0.1	+0.12	+0.12	+0.12	+0.12	+0.11	+0.11	+0.12	+0.12	+0.04	+0.00	+0.04	+0.00
	Δ 0.95	+0.18	+0.16	+0.17	+0.17	+0.11	+0.11	+0.17	+0.17	+0.04	+0.00	+0.04	+0.00
	p^*	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95
unix bge	Δ 0.1	+0.40	+0.38	+0.38	+0.38	+0.41	+0.35	+0.39	+0.41	+0.18	+0.02	+0.18	+0.02
	Δ 0.95	+0.21	+0.27	+0.25	+0.24	+0.24	+0.24	+0.20	+0.25	+0.16	+0.02	+0.16	+0.02
	p^*	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95
unix MiniLM	Δ 0.1	+0.35	+0.39	+0.39	+0.38	+0.39	+0.35	+0.33	+0.31	+0.02	+0.01	+0.02	+0.01
	Δ 0.95	+0.09	+0.24	+0.22	+0.16	+0.23	+0.18	+0.08	+0.09	-0.06	+0.01	-0.06	+0.01
	p^*	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	0.24	0.57	0.24	0.57
unix BM25	Δ 0.1	+0.11	+0.13	+0.13	+0.12	+0.09	+0.08	+0.09	+0.08	+0.03	+0.02	+0.04	+0.02
	Δ 0.95	-0.01	+0.06	+0.06	+0.05	+0.05	+0.05	-0.04	-0.09	+0.03	+0.02	+0.04	+0.02
	p^*	0.89	>0.95	>0.95	>0.95	>0.95	>0.95	0.82	0.45	>0.95	>0.95	>0.95	>0.95
webmasters bge	Δ 0.1	+0.16	+0.22	+0.22	+0.22	+0.17	+0.15	+0.16	+0.18	+0.02	+0.00	+0.02	+0.00
	Δ 0.95	-0.19	-0.01	-0.07	-0.12	-0.04	-0.05	-0.18	-0.15	-0.06	+0.00	-0.06	+0.00
	p^*	0.46	0.86	0.81	0.65	0.59	0.58	0.47	0.57	0.22	>0.95	0.22	>0.95
webmasters MiniLM	Δ 0.1	+0.32	+0.31	+0.31	+0.31	+0.26	+0.24	+0.31	+0.31	+0.11	+0.00	+0.11	+0.00
	Δ 0.95	+0.19	+0.27	+0.27	+0.26	+0.22	+0.20	+0.17	+0.16	+0.10	+0.00	+0.10	+0.00
	p^*	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95
webmasters BM25	Δ 0.1	+0.13	+0.13	+0.14	+0.14	+0.10	+0.06	+0.13	+0.13	+0.00	+0.00	+0.00	+0.00
	Δ 0.95	+0.16	+0.13	+0.14	+0.15	+0.14	+0.13	+0.16	+0.15	+0.00	+0.00	+0.00	+0.00
	p^*	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95	>0.95
mathematica bge	Δ 0.1	-0.00	+0.01	+0.01	+0.01	-0.00	+0.01	-0.00	-0.00	+0.00	+0.00	+0.00	+0.00
	Δ 0.95	-0.17	-0.08	-0.10	-0.14	-0.08	-0.08	-0.16	-0.17	+0.04	+0.04	+0.04	+0.04
	p^*	<0.10	0.20	0.23	0.22	<0.10	0.24	<0.10	<0.10	>0.95	>0.95	>0.95	>0.95
mathematica MiniLM	Δ 0.1	+0.32	+0.29	+0.29	+0.27	+0.35	+0.32	+0.32	+0.32	+0.07	+0.04	+0.07	+0.04
	Δ 0.95	-0.15	+0.12	+0.07	+0.00	+0.06	+0.04	-0.13	-0.16	-0.04	+0.04	-0.01	+0.04
	p^*	0.65	>0.95	>0.95	0.69	0.68	0.67	0.65	0.62	0.29	>0.95	0.29	>0.95
mathematica BM25	Δ 0.1	+0.05	+0.07	+0.07	+0.07	+0.10	+0.07	+0.05	+0.05	+0.03	+0.00	+0.03	+0.00
	Δ 0.95	-0.07	+0.03	+0.04	+0.01	-0.00	-0.00	-0.06	-0.07	+0.08	+0.00	+0.08	+0.00
	p^*	0.40	0.57	0.39	0.38	0.67	0.26	0.40	0.40	>0.95	>0.95	>0.95	>0.95